

Day 34 — Thu 8 Oct

Countdown: 2 days to CS Degree Paper-I

Method: start by recalling yesterday's topic with this page closed. Study the sections below, work every example by hand, then do 20 MCQs.

🎯 Degree — machine learning

Source: *ComputerScience_Degree/CSD_03_AI_and_MachineLearning.md*

Kernel K-means and hierarchical clustering

- **Kernel K-means:** map points into a higher-dimensional feature space (via a **kernel function**, e.g. RBF) and run K-means there, so it can find **non-linearly separable / non-spherical** clusters that plain K-means cannot. Uses the kernel trick — distances computed via the kernel without explicit mapping.
- **Hierarchical clustering:** builds a tree (**dendrogram**) of nested clusters.
- **Agglomerative (bottom-up):** start with each point as its own cluster, repeatedly merge the two closest. Linkage: single (min), complete (max), average.
- **Divisive (top-down):** start with one cluster, repeatedly split.
- No need to pre-choose k (cut the dendrogram at the desired level), but $O(n^2)$ or worse.

	K-means	Hierarchical
Need k in advance	Yes	No (cut dendrogram)
Output	Flat partition	Dendrogram (nested)
Complexity	$O(nk)$	$O(n^2 \log n)$ or $O(n^3)$
Re-run stability	Depends on init	Deterministic (given linkage)

Key points — the confusion matrix (binary)

	Predicted +	Predicted –
Actual +	TP (true positive)	FN (false negative)
Actual –	FP (false positive)	TN (true negative)

Metrics:

Metric	Formula	Meaning
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Overall correctness (misleading if imbalanced)
Precision	$TP/(TP+FP)$	Of those <i>predicted</i> positive, how many really are
Recall (Sensitivity, TPR)	$TP/(TP+FN)$	Of the <i>actual</i> positives, how many were caught
Specificity (TNR)	$TN/(TN+FP)$	Of actual negatives, how many caught
F1-score	$2 \cdot (P \cdot R) / (P + R)$	Harmonic mean of precision & recall (balances both)

Precision vs recall trade-off: raising the decision threshold usually raises precision but lowers recall, and vice versa. Use **precision** when false positives are costly (spam filter), **recall** when false negatives are costly (disease/fraud detection). **F1** balances the two.

Worked example

A model gives: **TP = 40, FP = 10, FN = 20, TN = 30**.

- Accuracy = $(40+30)/100 = 0.70$
- Precision = $40/(40+10) = 40/50 = 0.80$
- Recall = $40/(40+20) = 40/60 = 0.667$
- F1 = $2 \cdot (0.80 \cdot 0.667) / (0.80 + 0.667) = 2 \cdot (0.533) / (1.467) = 0.727$

🕒 Forensic — imaging, hashing, evidence

Source: *ComputerForensic/FOR_03_CyberCrimeAndIncidentResponse.md*

4.3 Order of volatility — memorise the list (RFC 3227)

Principle: collect evidence **from the most volatile to the least volatile**, because the act of collecting the less-volatile items (or simply the passage of time) destroys the more-volatile ones. Derived from **IETF RFC 3227, "Guidelines for Evidence Collection and Archiving"**.

The canonical ordered list (most → least volatile):

#	Evidence	Lifetime	Note
1	CPU registers, cache	Nanoseconds–ms	Effectively uncapturable in practice
2	Routing table, ARP cache, process table, kernel stats, live network connections	Seconds	Capture with live IR tools before they change/age out
3	Main memory (RAM) — running processes, open files, clipboard, encryption keys , injected/fileless malware, decrypted data	Until power off	The prize; capture first among the "collectable" items
4	Temporary file systems (swap/pagefile, tmpfs, <code>/tmp</code>)	Minutes–reboot	May hold fragments of memory/decrypted data
5	Disk / non-volatile storage (the drive image)	Persistent	The main dead-box exhibit
6	Remote logging & monitoring data relevant to the system	Depends on retention	Firewall/IDS/SIEM, server logs — grab before rotation
7	Physical configuration & network topology	Static	Diagram, cabling, device inventory
8	Archival media (backups, tapes, optical)	Long-lived	Least volatile; collect last

How to reproduce fast in the exam: *Registers/cache → ARP & routing & process tables & network connections → RAM → swap/temp files → disk → remote/monitoring logs → physical topology → archival backups*. Attach the reasoning ("collect volatile first because collecting stable evidence or time itself destroys volatile evidence") and cite **RFC 3227**.

Link to TP-I: this is the **Law of Progressive Change** (FOR_01 §1.5) made operational — everything changes with time; the rate differs; so collect the fastest-changing first.

5.4 Hashing — proving integrity

Concept: a **cryptographic hash** is a one-way function that maps any input to a fixed-length **digest**. Change one bit of the input and the digest changes completely (**avalanche effect**). So the hash is the **digital fingerprint** / **digital seal** of the evidence: compute it at acquisition, and recompute later to prove **nothing changed**.

Algorithm	Digest size	Status
MD5	128-bit (32 hex chars)	Fast; collision-broken (deliberate collisions can be produced). Still widely used <i>for integrity verification of evidence</i> where you are checking against accidental change, but not to prove uniqueness against a determined adversary.
SHA-1	160-bit (40 hex chars)	Collision-broken (SHAttered, 2017). Being retired.
SHA-256	256-bit (64 hex chars)	Part of SHA-2 ; currently recommended ; no practical collisions.
SHA-512 / SHA-3	512-bit / sponge	Also strong; SHA-3 (Keccak) is a different construction.

The workflow (write this as steps):

1. With a **write blocker** attached, compute the hash of the **source** device.
2. Acquire the **bit-stream image**.
3. Compute the hash of the **image**; it must equal the source hash → the copy is **verified**.
4. Record both hashes in the chain-of-custody/notes; treat the hash as the **seal**.
5. Do all analysis on a **working copy**; before and after analysis, re-hash to prove the evidence copy is unchanged. Any mismatch must be **investigated and explained** (recall the SSD caveat — FOR_02 §2.2 — where garbage collection can legitimately change an SSD's hash between reads).

Collisions — what to say:

- A **collision** = two *different* inputs with the *same* hash. MD5 and SHA-1 have **practical, engineered collisions**; SHA-256 does not.
- **Why it does not automatically destroy MD5's evidentiary use:** producing a collision requires *crafting both files*; it does **not** let an adversary alter your seized evidence to match a given hash (that is a far harder **pre-image/second-pre-image** attack, still infeasible even for MD5). Best practice today: **compute two hashes (e.g. MD5 + SHA-256)** so a single broken algorithm cannot be argued to undermine integrity. Say exactly this if asked "is MD5 still acceptable?" — nuance scores.

Digital analogue of the seal (link to FOR_01): the hash is to a disk image what a tamper-proof seal + panchnama is to a physical exhibit — it proves the object produced in court is the object seized, unaltered.

§5.5 Likely exam questions

Marks	Question
5	Distinguish between a bit-stream image and a logical copy.
5	What is a write blocker and why is it essential?
5	What is a hash value? Name three hash algorithms with their digest sizes.
5	What is a hash collision? Is MD5 still usable for evidence?
10	Describe the process of forensically imaging and verifying a hard disk.

10	Compare raw (dd), E01 and AFF image formats.
15	Explain how the integrity of digital evidence is established and maintained from seizure to court, covering write blockers, imaging formats, hashing and the §65B/§63 certificate.

MCQ traps

- **MD5 = 128-bit; SHA-1 = 160-bit; SHA-256 = 256-bit.** Constantly swapped.
- **Logical copy cannot recover deleted files;** bit-stream can.
- **dd/raw has no embedded hash/metadata; E01 does.**
- A hash change means the data changed (unless a documented SSD-GC / drive-firmware reason applies).
- Write blocker blocks **writes to the evidence**, allowing reads.

§6. Recovery of Deleted, Hidden and Altered Files

6.2 File carving — the technique

Concept: carving reconstructs files **from content alone**, using **header (magic number)** and often **footer** signatures and internal structure, without any file-system metadata.

- **Header/footer carving:** find a known header (e.g. JPEG **FF D8 FF**) and the matching footer (JPEG **FF D9**), extract everything between. Works for contiguous files.
- **Fragment recovery / SmartCarving:** handles **fragmented** files by reassembling fragments — much harder; needs structure validation.
- **Limitations: fragmentation** defeats naive carving; a carved file has **no original name, path or timestamps** (those live in metadata); overwritten regions are unrecoverable; on **TRIM-enabled SSDs** the data is often already gone (FOR_02 §2.2). Tools: **PhotoRec/Scalpel/Foremost**, and the carving engines inside Autopsy/EnCase/FTK.

6.3 File signature vs extension table — memorise this (guaranteed MCQ + practical answer)

Concept: the **extension** (**.jpg**, **.pdf**) is just part of the *name* and is **trivially changed** to hide a file; the **signature (magic number)** is bytes **inside** the file that reveal its **true type**. **File-signature analysis** = comparing the header bytes to the extension to detect **extension mismatch/masquerading** (e.g. a **secret.jpg** that is really a RAR archive, or an **.exe** renamed to **.txt**).

File type	Extension	Header (magic) bytes (hex)	ASCII / note	Footer (if any)
JPEG	.jpg/.jpeg	FF D8 FF (E0/E1/EE...)	ÿØÿ	FF D9
PNG	.png	89 50 4E 47 0D 0A 1A 0A	.PNG....	49 45 4E 44 AE 42 60 82 (IEND)
GIF	.gif	47 49 46 38 37 61 or 47 49 46 38 39 61	GIF87a / GIF89a	00 3B

PDF	.pdf	25 50 44 46	%PDF	25 25 45 4F 46 (%EOF)
ZIP / DOCX / XLSX / PPTX / JAR / APK / ODF	.zip etc.	50 4B 03 04 (also 50 4B 05 06 empty, 50 4B 07 08 spanned)	PK..	—
RAR	.rar	52 61 72 21 1A 07 00 (v4) / ...01 00 (v5)	Rar!..	—
7-Zip	.7z	37 7A BC AF 27 1C	7z¼~'.	—
GZIP	.gz	1F 8B	—	—
Windows PE (EXE/DLL)	.exe/.dll	4D 5A	MZ (Mark Zbikowski)	—
ELF (Linux exe)	—	7F 45 4C 46	.ELF	—
PDF-vs-Office note			modern Office = ZIP (PK); legacy Office (.doc/.xls/.ppt) = D0 CF 11 E0 A1 B1 1A E1 (OLE/CFBF)	
MP3	.mp3	49 44 33 (ID3 tag) or FF FB frame sync	—	—
MP4/MOV	.mp4	 66 74 79 70 (ftyp at offset 4)	—	—
PST/MS Outlook	.pst	21 42 44 4E (!BDN)	—	—
SQLite (many app DBs)	.db	53 51 4C 69 74 65 20 66 6F 72 6D 61 74 20 33 00	SQLite format 3	—

The five you must be able to write from memory: JPEG FF D8 FF, PDF %PDF (25 50 44 46), PNG 89 50 4E 47, ZIP/PK 50 4B 03 04, EXE MZ (4D 5A), plus GIF GIF87a/ GIF89a. These appear verbatim in MCQs and in the "how would you detect a file hiding its type" descriptive answer.